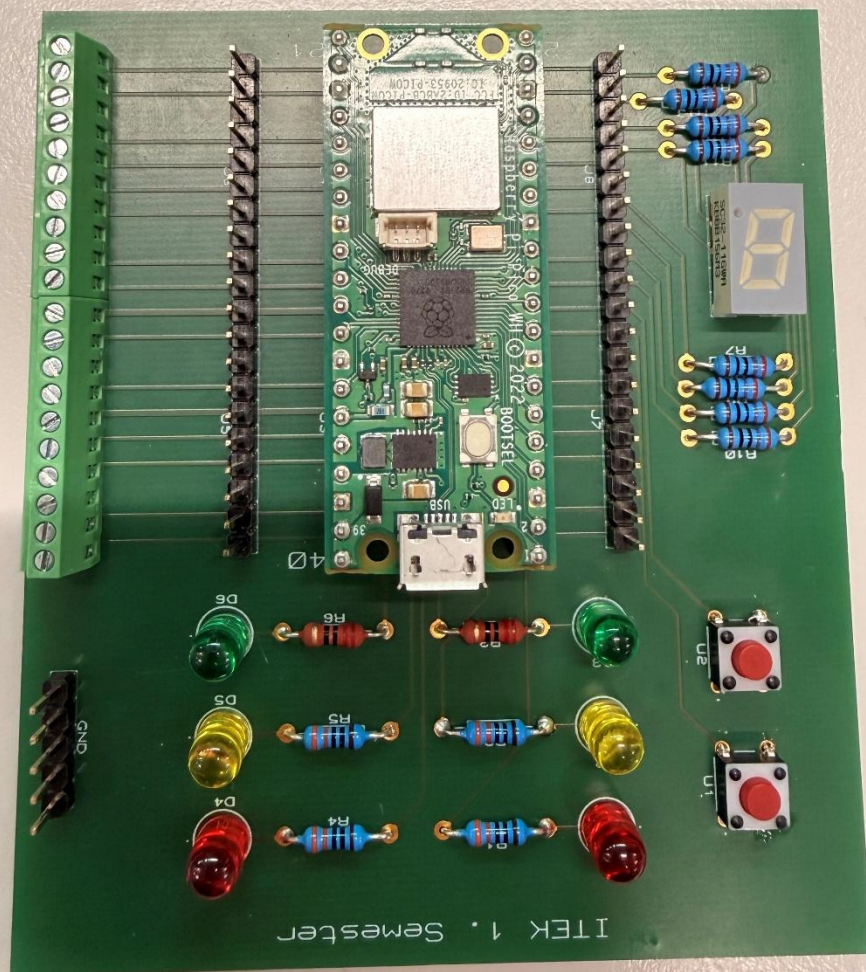


Embedded bibel for ITEK



Indhold

Figur oversigt.....	5
Historisk tidslinje.....	6
Chip-fabrikanter (producerer/støber chippene).....	6
Chip-leverandører 2021 (efter salg)	6
Praktisk tjekliste	7
Multimeter og voltmeter	8
Eksempel: simpelt konsol-voltmeter	8
Hardware: Pico 2 W og printet	9
Raspberry Pi Pico 2 W — nøgletal.....	9
Pin-layout for LED'erne	11
Pin-layout for Knapperne.....	12
Sådan sættes det i MicroPython	12
PULL_UP: knappen forbundet til stel	12
PULL_UP: knappen forbundet til 3,3 V	13
Pin-layout for 7-segment display	14
Pin-layout for ADC	15
Pulse Width Modulation (PWM) teori	16
Duty cycle (arbejdscyklus)	16
PWM: Spændingsregulering	16
Duty cycle visualiseret	17
Raspberry Pi Pico — PWM-kanaler	17
Eksempel: åndende LED (fade ind og ud).....	18
Eksempel: lysstyrke styret af et potentiometer (ADC + PWM)	18
Mål duty cycle med et oscilloskop	19
ADC og spændingsdeler teori	20
Sådan tænker du en spændingsdeler.....	20
Statemachines teori	21
Eksempel: hysteres ved en termostat	21
Eksempel på en statemachine:.....	22

Farvet konsoloutput (ANSI escape-koder)	23
Random statemachine	23
Eksempel: random_statemachine_random_getint_style.py	24
7-segment display teori	26
Eksempel 1: cifferopslag som separat modul.....	26
Eksempel 2: tænd pinene for cifret 0	26
GY-53 ToF-afstandssensor teori	27
Netværkssværktøjer	28
Bageopskrift med PWM-eksempel	29
1. Importér det du skal bruge	29
2. Opret dine pins.....	29
3. Sæt PWM-frekvensen	29
4. Skriv selve løkken	29
5. Det færdige program	30
Variation 1: brug en knap i stedet for tid	30
Variation 2: flere "faser" (trafiklys, elevator, hysteres).....	30
Genbrugelige skabeloner	31
1. Input med validering	31
2. Statemachine	31
Eksempel 1: Forenklet trafiklys, som statemachine	32
3. ADC og spænding.....	33
Eksamen, points og svar	34
Ohms lov	38
Ohms lov formler	38
Modstand i Serieforbindelse.....	38
Modstand i Parallelforbindelse	38
Modstand i Serie- og Parallelforbindelse	39
Ohms lovs trekanter	39
Oversigt over ohms love	40
Præfiks	40

Enheder til Ohms lov	41
Hvad siger Ohms lov?.....	41
Ohms lov Formel	41
Sådan taster du det på lommeregneren	42
Eksempler på ohms lov	42
Eksempel 1: find spændingen (U)	42
Eksempel 2: find strømmen (I)	43
Eksempel 3: find modstanden (R)	43
Eksempel 4: Find strømmen (I) 2.....	44
Eksempler på modstand i serieforbindelse, parallelforbindelse og serie- og parallelforbindelse	44
Serieforbindelse	44
Eksempel: 2 modstande i serie	45
Modstand i parallelforbindelse	46
Eksempel — to modstande i parallel.....	46
Modstand i serie- og parallelforbindelse	47
Figur a: serieforbindelse inde i en parallelforbindelse	47
Eksempel: figur a med tal.....	48
Figur b: parallelforbindelse inde i en serieforbindelse	48
Sådan taster du Ohms lov ind på lommeregneren.....	49

Figur oversigt

Figure 1: Diagram over bbb	9
Figure 2: 7 segment display with GPIO	14
Figure 3: Duty cycle graf.....	17
Figure 4: Diagram for analog til digital konvertering	20
Figure 5: Hysteresese sløjfe	21
Figure 6: Diagram for Random Statemachine	24
Figure 7: Modstand i Serieforbindelse	38
Figure 8: Modstand i Parallelforbindelse	38
Figure 9: Modstand i Serie- og Parallelforbindelse.....	39
Figure 10: ohms love trekanter	39
Figure 11: oversigt over ohms lov figur	40
Figure 12: præfiks	40
Figure 13: Huskefigur: dæk det, du søger, til med fingeren. De to andre bogstaver viser, om du skal gange eller dividere.....	42
Figure 14: Modstand i serieforbindelse.....	45
Figure 15: modstand i parallelforbindelse	46
Figure 16: Figur a, Serieforbindelse inde i en parallelforbindelse	47
Figure 17. Figur b, parallelforbindelse inde i en serieforbindelse	48

Historisk tidslinje

År	Hændelse
1971	Intel 4004: verdens første mikroprocessor, ca. 175 kHz.
1974-1976	Intel 8080 og 8085 — tidlige 8-bit processorer, 3–6 MHz, 8-bit databus, 16-bit adressebus. Zilog Z80 — konkurrent til 8085, brugt i mange klassiske hjemmecomputere.
2000	Intel Pentium 4 — ca. 1,8 GHz.
2026	AMD/Intel over 3 GHz, mange kerner.

MOORES LOV

Antallet af transistorer på en chip fordobles ca. hvert 2. år. Det er en empirisk observation (baseret på historiske data) – ikke en fysisk lov som tyngdekraften.

Chip-fabrikanter (producerer/støber chippene)

TSMC (Taiwan) og Samsung (Sydkorea) dominerer markedet.

Chip-leverandører 2021 (efter salg)

Samsung, Intel, SK Hynix, Micron, Qualcomm, Nvidia, Broadcom, Texas Instruments, MediaTek og AMD.

Praktisk tjekliste

- ☐ **RYD OP EFTER DIG:** Skriv `try / except KeyboardInterrupt`: rundt om hovedløkken og nulstil LED/PWM i except-blokken. Det går igen i alle eksempler, så det forventes formentlig igen.
- ☐ **PULL_UP = TRYK ER 0:** Den hyppigste fejl i knap-opgaver. Skriv det på hånden, hvis du er nervøs for at glemme det.
- ☐ **DIGITAL VS. PWM:** Brug `led.on()` / `off()` til ren tænd/sluk. Brug `pwm.duty_u16(...)` når opgaven nævner "lysstyrke" eller "dæmpe".
- ☐ **TEGN FØR DU KODER:** Ved statemachine-opgaver: tegn cirkler og pile først. Hver cirkel = én funktion i koden.
- ☐ **TEST ADC MED POTMETER:** Test din `u16`→volt-formel med et potmeter, før du stoler på en rigtig sensor. Hurtigste måde at opdage fejl i formlen.
- ☐ **65535, IKKE 4095:** MicroPythons `read_u16()` skalerer altid op til 16-bit, selvom ADC'en fysisk er 12-bit. Del derfor altid med 65535.

Multimeter og voltmeter

Et multimeter kan måle tre ting:

Måling	Beskrivelse
Spænding	Potentialeforskel mellem to punkter.
Strøm	Hvor meget strøm der løber gennem kredsløbet.
Modstand	Bruges til at teste komponenter/kredsløb.

Et voltmeter bygget med Pico'en er i praksis: ADC + en formel + en måde at vise resultatet på (konsol-print, LEDbargraf eller 7-segment)

Eksempel: simpelt konsol-voltmeter

```
from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(26))

try:
    while True:
        adc_voltage = adc.read_u16() / 65535 * 3.3
        print(f"Spænding: {adc_voltage:.2f} V")
        sleep(0.5)
except KeyboardInterrupt:
    print("Stoppet.")
```


Hardware: Pico 2 W og printet

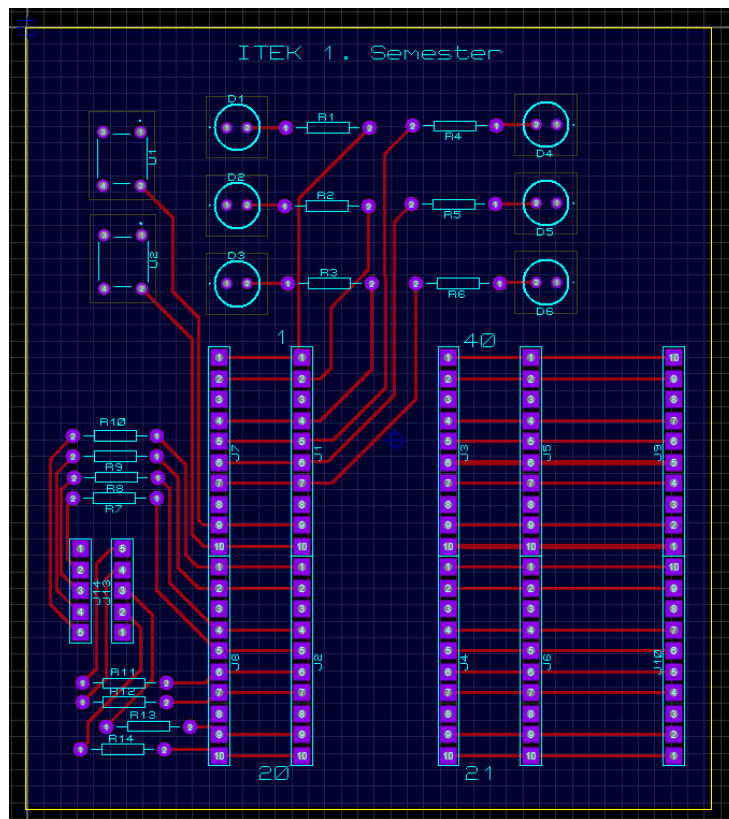


Figure 1: Diagram over bbb

Raspberry Pi Pico 2 W — nøgletal

Egenskab	Beskrivelse
Chip	RP2350, dual-core (Arm Cortex-M33 @ 150 MHz eller valgfrit Hazard3 RISC-V)
Hukommelse	520 kB SRAM, 4 MB flash
Trådløs	2,4 GHz WiFi 802.11n + Bluetooth 5.2 (Infineon CYW43439-chip)
I/O	26 multifunktions-GPIO-ben, 3×12-bit ADC, SWD-debugport, TrustZone

VÆR OPMÆRKSOM

Pico 2 W kører MicroPython direkte på "bare metal" – ligesom den oprindelige Pico. ADC'en er fysisk 12-bit, men `read_u16()` skalerer altid op til 16-bit (0-65535) – derfor deler man med 65535, ikke 4095, i koden.

Antal	Del
1	Raspberry Pi Pico 2 W
2	Rød LED-indikator (Würth Elektronik 151051RS11000)
2	Gul LED-indikator (Würth Elektronik 151051YS04000)
2	Grøn LED-indikator (Würth Elektronik 151051VS04000)
4	330 Ω modstand (Yageo MFR-25FRF52-330R)
2	220 Ω modstand (KOA Speer CF1-2CT52R221J)
2	Trykknop (Adafruit 367)
2	Phoenix Contact MKDS 1/10-3,5-serien printklemme

VÆR OPMÆRKSOM

330 Ω bruges typisk til de 4 trafiklys-LED'er, mens 220 Ω passer bedre til 7-segment-displayets segmenter. Lavere modstand giver mere strøm/lys pr. segment – nødvendigt fordi hvert segment er en lille LED, der skal være tydeligt synlig på afstand.

Pin-layout for LED'erne

General-purpose input/output (GPIO) for led'erne på Big Beautiful Board.

Variable	GPIO	Beskrivelse
ledRL	0	Rød led venstre
ledYL	1	Gul led venstre
ledGL	2	Grøn led venstre
ledRR	3	Rød led højre
ledYR	4	Gul led højre
ledGR	5	Grøn led højre

```
ledRL = Pin (0, Pin.OUT)
ledYL = Pin (1, Pin.OUT)
ledGL = Pin (2, Pin.OUT)
ledRR = Pin (3, Pin.OUT)
ledYR = Pin (4, Pin.OUT)
ledGR = Pin (5, Pin.OUT)
```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

Pin-layout for Knapperne

Når en knap er tilsluttet et GPIO-ben på en mikrocontroller, skal benet vide, hvad det skal "falde tilbage til", når knappen ikke er trykket. Det er det, pull-up og pull-down klarer.

Egenskab	Beskrivelse
Pull-up	Et ben på μ C'en bliver trukket op til 3,3 V
Pull-down	Et ben på μ C'en bliver trukket til stel (0 V)

VÆR OPMÆRKSOM

Uden pull-up eller pull-down kan et ben "flyde" (floating) – det ved ikke, om det skal læses som 0 eller 1, når knappen ikke er trykket, og kan derfor skifte tilstand tilfældigt. Pull-up/pull-down sikrer, at benet altid har en defineret tilstand, når kredsløbet står åbent.

Sådan sættes det i MicroPython

General-purpose input/output (GPIO) for knapperne på Big Beautiful Board.

Variable	GPIO	Beskrivelse
Btn1	6	Fysisk knap
Btn2	7	Fysisk knap

Kodeblok for knapperne med PULL_UP.

```
Btn1 = Pin (6, Pin.IN, Pin.PULL_UP)
Btn2 = Pin (7, Pin.IN, Pin.PULL_UP)
```

PULL_UP: knappen forbundet til stel

```
Knap = Pin(2, Pin.IN, Pin.PULL_UP)
```

Bruges, når knappen er tilsluttet stel på Pico'en i den ene side, og en GP-port i den anden side (sådan er knappen på jeres print forbundet).

Knap-tilstand	MCU-indgang
S1 ikke sluttet (ikke trykket)	Vcc → logisk 1
S1 sluttet (trykket)	Stel → logisk 0

PULL_UP: knappen forbundet til 3,3 V

```
Knap = Pin(2, Pin.IN, Pin.PULL_DOWN)
```

Bruges, når knappen i stedet er tilsluttet **3,3 V** på Pico'en.

Knap-tilstand	MCU-indgang
S1 ikke sluttet (ikke trykket)	Vcc → logisk 1
S1 sluttet (trykket)	Stel → logisk 0

VÆR OPMÆRKSOM

De to opsætninger giver modsat logik. Med PULL_UP er tryk = 0. Med PULL_DOWN er tryk = 1. Tjek altid, hvordan knappen faktisk er forbundet på dit print, før du antager hvilken af de to du skal bruge.

KILDE: Pull up and DOWN.pptx

Læs mere i Pull up and DOWN.pptx

Pin-layout for 7-segment display

Variable	GPIO	Position	Segm.
Led1	8	bund	d
Led2	9	bund-venstre	e
Led3	10	midt	g
Led4	11	top-venstre	f
Led5	12	top	a
Led6	13	top-højre	b
Led7	14	bund-højre	c
Led8	15	punktum	dp

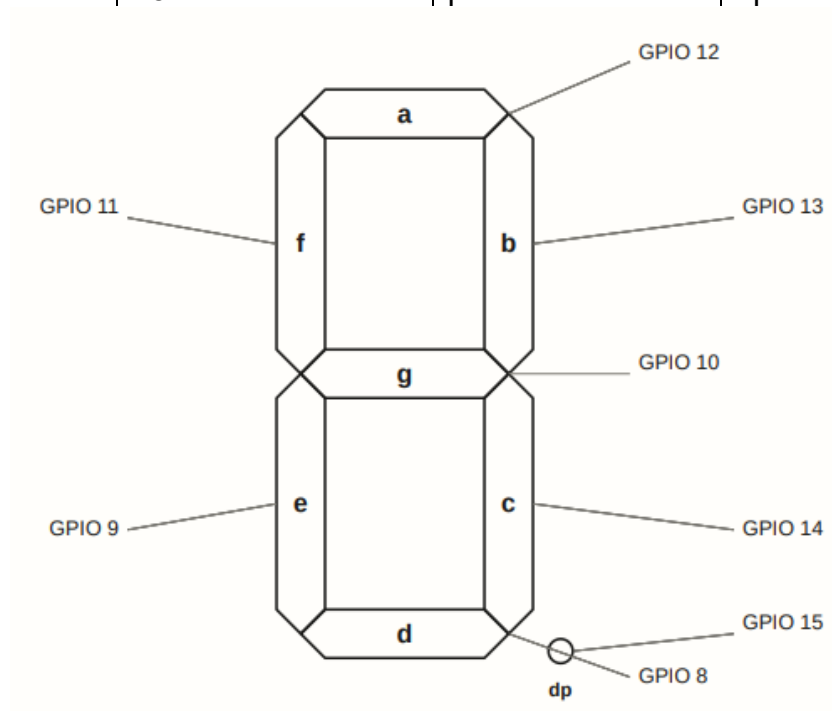


Figure 2: 7 segment display with GPIO

Pin-layout for ADC

ADC-indgangen (potmeter/spænding) sidder fast på Pin (26) (ADC0).

```
from machine import  
ADC pot = ADC (Pin (26))
```

VÆR OPMÆRKSOM

Pin.PULL_UP betyder, at knappen læses som 0, når den er trykket ned – ikke 1. Tænk det som "normalt trukket op til 1 (sluppet), tryk trækker ned til 0".

Pulse Width Modulation (PWM) teori

PWM (Pulse Width Modulation) regulerer lysstyrke/effekt ved hurtigt at tænde og slukke et signal. Duty cycle er forholdet (i %) mellem "on"-tid og "off"-tid indenfor én periode — 50% duty cycle betyder tændt halvdelen af tiden.

VÆR OPMÆRKSOM

PWM-carrier-frekvensen (`pwm.freq(1000)`) er den interne, hurtige switching-frekvens, der bruges til selve dæmpningen. Den er ikke det samme som en blinkfrekvens, du selv styrer med `sleep()` i en `while`-løkke.

Duty cycle (arbejdscyklus)

Arbejdscyklussen beregnes som forholdet mellem on-tiden og den samlede periode. Ved at bruge den tidligere beregnede periode beregnes arbejdscyklussen som:

PWM: Spændingsregulering

PWM-signalet giver en varierende spænding ved forskellige duty cycles. Denne metode anvendes inden for flere områder, såsom:

- **Switching-regulatorer**
- **LED-dæmpere**
- **Lydsignaler**
- **Generering af analoge signaler**
- **...og mange flere**

Spændingsregulering opnås ved at udjævne PWM-signalet. Udgangsspændingen beregnes med følgende formel:

Som det fremgår af formlen, kan udgangsspændingen direkte varieres ved at ændre **Ton**-værdien.

- Hvis **Ton** er 0, er **Vout** også 0.
- Hvis **Ton** er lig med den samlede periode **Ttotal**, er **Vout** lig med **Vin** (maksimal spænding),

Duty cycle visualiseret

Jo højere duty cycle, jo tættere ligger den gennemsnitlige ("den værdi du ser") spænding på MAX. Perioden er tiden for én hel on/off-cyklus; duty er andelen af perioden, hvor signalet er højt.

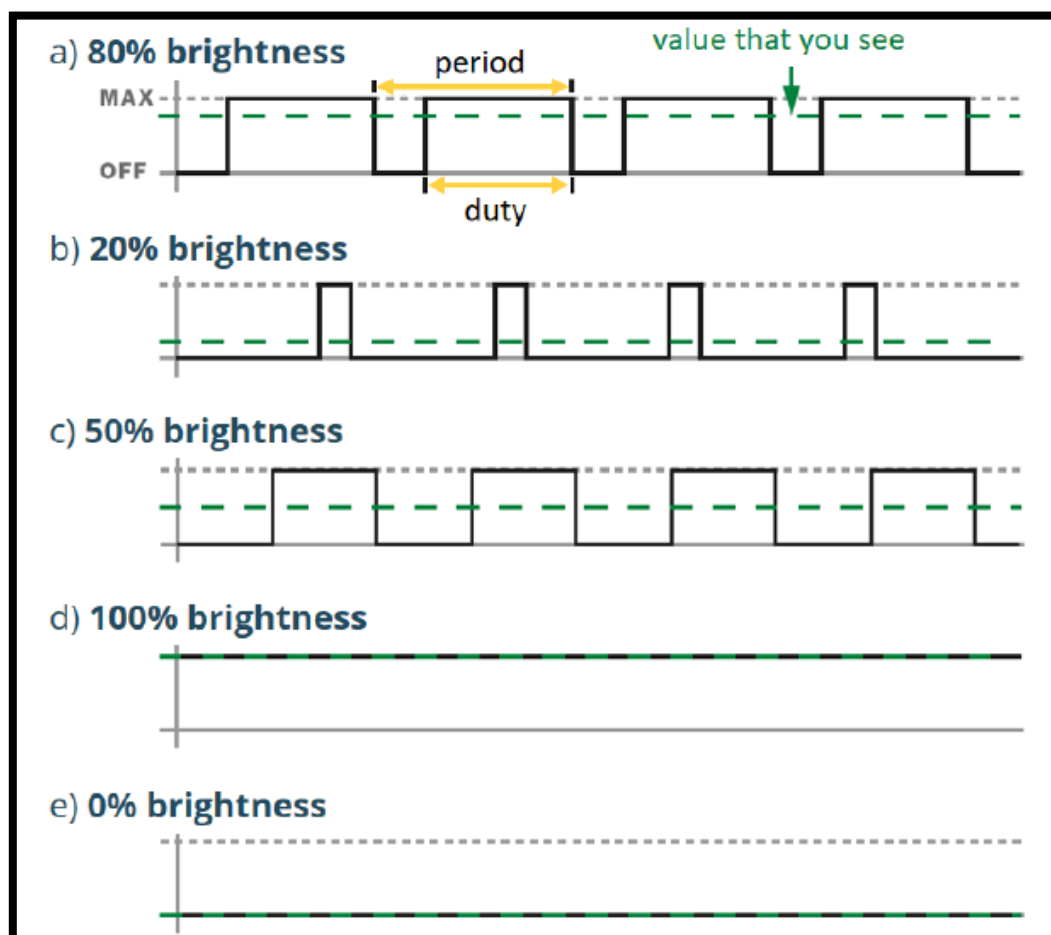


Figure 3: Duty cycle graf

Øjet/lyssensoren opfatter ikke de hurtige tænd/sluk-pulser enkeltvis — det du ser, er gennemsnittet af signalet over tid. Det er præcis derfor PWM virker som "analog" lysstyrke-regulering, selvom signalet reelt kun er digitalt tændt eller slukket.

Raspberry Pi Pico — PWM-kanaler

Egenskab	Beskrivelse
Antal slices	8 uafhængige PWM-generatorer ("slices"), hver med 2 kanaler = 16 kanaler i alt
Frekvensområde	Ca. 8 Hz til 62,5 MHz (mikrocontrolleren kører internt ved 125 MHz)

Delt frekvens	De to kanaler i samme slice kører altid ved samme frekvens, men kan have hver sin duty
---------------	--

Eksempel: åndende LED (fade ind og ud)

Her styrer den ydre sleep(0.02)-løkke, hvor hurtigt lyset stiger/falder (det du kan se), mens pwm.freq(1000) er den usynlige, interne switching-frekvens.

```
from machine import Pin, PWM
from time import sleep led = PWM(Pin(2))

led.freq(1000)

try:
    while True:

        for duty in range(0, 65536, 2000): # fade ind
            led.duty_u16(duty)
            sleep(0.02)

        for duty in range(65535, -1, -2000): # fade ud
            led.duty_u16(duty)
            sleep(0.02)

except KeyboardInterrupt:
    led.duty_u16(0)
    led.deinit()
```

Eksempel: lysstyrke styret af et potentiometer (ADC + PWM)

En meget almindelig opgave: brug et potentiometer til at style en LED's lysstyrke i realtid. Fordi adc.read_u16() og pwm.duty_u16() begge arbejder i samme talområde (0–65535), kan du sende ADCværdien direkte videre til PWM'en — uden at skulle regne den om.

```
from machine import Pin, PWM, ADC
from time import sleep

led = PWM(Pin(2)) # ledGL fra dit print
pot = ADC(Pin(26)) # potentiometer på ADC0

led.freq(5000)

try:
    while True:
```

```
pot_value = pot.read_u16()
# Slå helt fra ved meget lave værdier, så LED'en
# ikke "gløder" svagt pga. støj tæt på 0

if pot_value <= 420:
    led.duty_u16(0)

else:
    led.duty_u16(pot_value)
    sleep(0.1)

except KeyboardInterrupt:
    led.duty_u16(0)
    led.deinit()
```

VÆR OPMÆRKSOM

Et potentiometer giver sjældent en helt ren 0 i sin nederste ende – der kan ligge et par hundrede "støjende" enheder, som ellers ville få LED'en til svagt at gløde, selvom den burde være helt slukket. En simpel tærskel (her 420) løser det.

Mål duty cycle med et oscilloskop

Vil du se PWM-signalet med egne øjne i stedet for at regne det ud, kan du bruge et oscilloskop med to probe'r:

- Den ene probe sættes på selve PWM-benet (fx GPIO 2).
- Den anden probe (eller probens jordklemme) sættes på GND, så du har et fælles referencepunkt.

Skoperet tegner signalet som en firkantbølge, hvor du direkte kan aflæse periodens længde (og dermed frekvensen) samt hvor stor en andel af perioden der er høj (duty cyclen), en god måde at bekræfte, at koden faktisk gør det, du tror den gør.

ADC og spændingsdeler teori

En ADC (Analog-til-Digital Converter) omdanner et analogt spændingssignal til en digital værdi, mikrocontrolleren kan regne videre på. Pico'ens ADC-indgang (GPIO 26 / ADC0) måler kun 0–3,3 V.

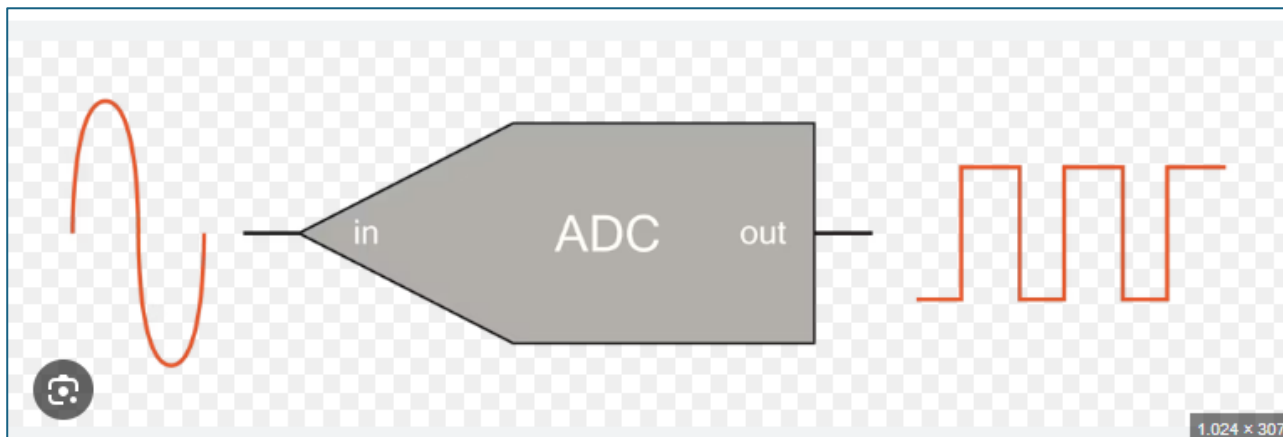


Figure 4: Diagram for analog til digital konvertering

VÆR OPMÆRKSOM HVIS DU SKAL MÅLE MERE END 3,3 V

Fx et 6,4–8,4 V robotbatteri eller 0–12 V, så skal du bruge en spændingsdeler (to modstande) FØR signalet rammer ADCindgangen, og regne den rigtige spænding ud igen i softwaren.

Sådan tænker du en spændingsdeler

Sæt to modstande R1 og R2 i serie fra din høje spænding ned til GND, og mål på punktet mellem dem. Formlen for hvad ADC'en reelt ser er:

```
adc_voltage = adc.read_u16() / 65535 * 3.3  
real_voltage = adc_voltage / (R2 / (R1 + R2))
```

Jo større R1 er i forhold til R2, jo mere "skalerer" du den høje spænding ned til noget, Pico'en kan tåle — men husk altid at gange tilbage i koden for at få den rigtige værdi at vise.

Statemachines teori

En statemachine strukturerer en proces som en række states (tegnes som cirkler), med pile der viser hvilken state man skifter til og hvorfor. I koden svarer hver cirkel til én funktion, som udfører sit arbejde og returnerer (ikke kalder) den næste state-funktion.

KILDE: Statemachine v2.pptx

Læs mere i Statemachine v2.pptx

Eksempel: hysteresese ved en termostat

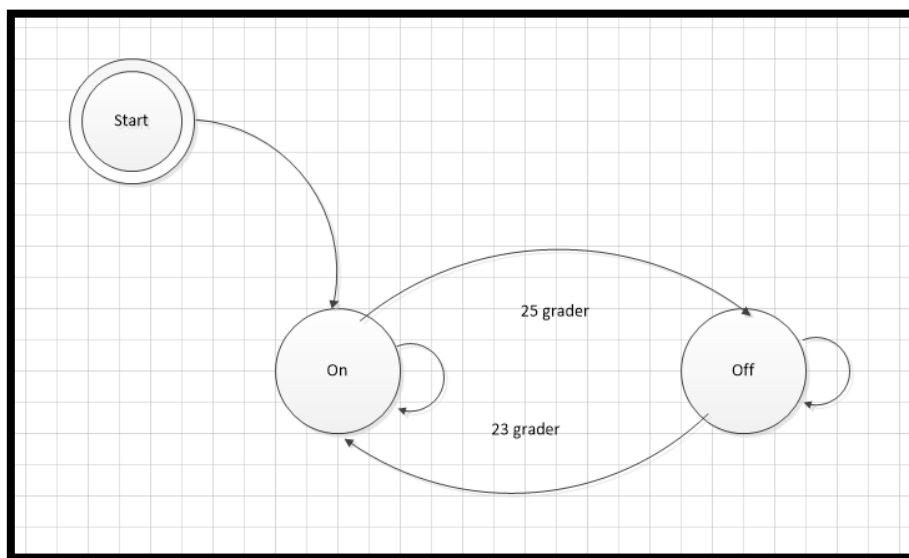


Figure 5: Hysteresese sløjfe

Uden hysteresese ville en termostat "flimre" tændt/slukket lige omkring ét sætpunkt, fx 20°C. Med hysteresese bruges to grænser i stedet:

```
ON_GRÆNSE = 19 # tænd varmen, hvis temperatur falder under denne
OFF_GRÆNSE = 21 # sluk varmen, når temperatur stiger over denne

def state_off():
    if temperatur() < ON_GRÆNSE:
        return state_on
    return state_off

def state_on():
    if temperatur() > OFF_GRÆNSE:
        return state_off
```

```
return state_on
```

Fordi de to grænser er forskellige (19 og 21, ikke begge 20), skifter systemet ikke frem og tilbage for hvert lille sving omkring 20°C.

Eksempel på en statemachine:

```
from machine import Pin
from time import sleep

def get_int(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))
            if low <= value <= high:
                return value
            print(f"Enter a value between {low} and {high}.")
        except ValueError:
            print("Please type a whole number.")

ledRL = Pin(0, Pin.OUT)
ledYL = Pin(1, Pin.OUT)
ledGL = Pin(2, Pin.OUT)

ledRR = Pin(3, Pin.OUT)
ledYR = Pin(4, Pin.OUT)
ledGR = Pin(5, Pin.OUT)

groen_tid = get_int("Enter a 1-20 seconds for green led: ", 1, 20)
gul_tid = get_int("Enter a 1-20 seconds for yellow led: ", 1, 10)
roed_tid = get_int("Enter a 1-20 seconds for red led: ", 1, 20)

def alle_slukket():
    ledRL.value(0)
    ledYL.value(0)
    ledGL.value(0)

    ledRR.value(0)
    ledYR.value(0)
    ledGR.value(0)

def state_groen():
    alle_slukket()
    ledGL.value(1)
    ledGR.value(1)
    print("Green... ready...")
    sleep(groen_tid)
    return state_gul
```

```

def state_gul():
    alle_slukket()
    ledYL.value(1)
    ledYR.value(1)
    print("Yellow... ready...")
    sleep(gul_tid)
    return state_roed

def state_roed():
    alle_slukket()
    ledRL.value(1)
    ledRR.value(1)
    print("Red... ready...")
    sleep(roed_tid)
    return state_groen

try:
    state = state_groen          # initial state
    while state:
        state = state()         # kør statemachine
except KeyboardInterrupt:
    alle_slukket()
    print("Stoppet, alle LED'er slukket.")

```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

Farvet konsoloutput (ANSI escape-koder)

Nyttigt hvis en opgave vil have fx "grønt lys = grøn tekst" i konsollen:

```

print('\x1b[7;32;40m' + "Grøn! Kør" + '\x1b[0m') # grøn baggrund/tekst
print('\x1b[6;30;43m' + "Gul! stop" + '\x1b[0m') # gul
print('\x1b[6;30;41m' + "Rød! stop" + '\x1b[0m') # rød

```

Formatet er \x1b[;;m og nulstilles altid med \x1b[0m bagefter — ellers "smitter" farven resten af outputtet.

Random statemachine

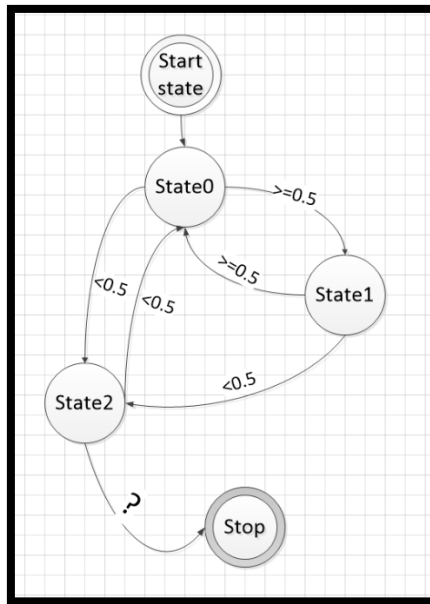


Figure 6: Diagram for Random Statemachine

Eksempel: random_statemachine_random_getint_style.py

```

from machine import Pin
from random import random
from time import sleep

led0 = Pin(0, Pin.OUT)
led1 = Pin(1, Pin.OUT)
led2 = Pin(2, Pin.OUT)

def get_int(prompt, lo, hi):
    while True:
        try:
            val = int(input(prompt))
            if lo <= val <= hi:
                return val
            print(f"Enter a value between {lo} and {hi}.")
        except ValueError:
            print("Please type a whole number.")

step_time = get_int("Seconds per state (1-5): ", 1, 5)
stop_chance_pct = get_int("Chance to stop at State2, in % (0-100): ", 0, 100)
stop_threshold = stop_chance_pct / 100

def all_off():
    led0.off(); led1.off(); led2.off()

def state0():
    all_off(); led0.on()
    sleep(step_time)
    return state1 if random() >= 0.5 else state2

```



```

def state1():
    all_off(); led1.on()
    sleep(step_time)
    return state0 if random() >= 0.5 else state2

def state2():
    all_off(); led2.on()
    sleep(step_time)
    if random() < stop_threshold:
        return None
    return state0

try:
    state = state0
    while state:
        state = state()
    all_off()
    print("Stopped at State2.")
except KeyboardInterrupt:
    all_off()
    print("Stopped, outputs cleared.")

```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

7-segment display teori

Består af 7 uafhængige LED'er (segmenterne a–g), der tændes i forskellige kombinationer for at danne cifre 0–9, plus evt. et 8. segment (decimalpunktet). Hvert segment skal have sin egen 220 Ω modstand for ikke at trække for meget strøm.

VÆR OPMÆRKSOM

Gem cifferkombinationerne (0–9) i et separat Python-modul, du kan importere, i stedet for at skrive dem ind i hovedprogrammet hver gang – nemmere at genbruge og nemmere for censor at læse.

Eksempel 1: cifferopslag som separat modul

```
# digits.py – importeres i hovedprogrammet
# Rækkefølge: (a, b, c, d, e, f, g)

DIGITS = {
    0: (1,1,1,1,1,1,0),
    1: (0,1,1,0,0,0,0),
    2: (1,1,0,1,1,0,1),
    # ... osv. for 3-9
}
```

Eksempel 2: tænd pinene for cifret 0

```
from machine import Pin

led1 = Pin(8, Pin.OUT) # d
led2 = Pin(9, Pin.OUT) # e
led3 = Pin(10, Pin.OUT) # g
led4 = Pin(11, Pin.OUT) # f
led5 = Pin(12, Pin.OUT) # a
led6 = Pin(13, Pin.OUT) # b
led7 = Pin(14, Pin.OUT) # c
led8 = Pin(15, Pin.OUT) # dp
pins = [led5, led6, led7, led1, led2, led4, led3, led8] # a,b,c,d,e,f,g,dp
digit_0 = (1,1,1,1,1,1,0,0)

for pin, value in zip(pins, digit_0):
    pin.value(value)
```

KILDE: 7 segment.pptx

Læs mere i 7 segment.pptx

GY-53 ToF-afstandssensor teori

Time-of-Flight-sensor (baseret på VL53L0X-chippen), måleområde 0–2 meter. Kan sende data på tre måder: UART, PWM eller I²C.

I PWM-mode sender sensoren et signal med fast periode (~22 ms), og pulslængden (HIGH-tiden) er proportional med afstanden:

$\text{Afstand (mm)} = \text{Pulslængde } (\mu\text{s}) / 10$

VÆR OPMÆRKSOM

Fordel: simpel implementering, ingen adressering nødvendig. Ulempe: mindre robust mod støj, kræver præcis timing – derfor bruges `time.ticks_us()` og `ticks_diff()`, ikke almindelig `time.time()`.

Netværksværktøjer

Værktøj	Bruges til
Putty	SSH/Telnet-klient — bruges til at logge ind på en Raspberry Pi/enhed over netværket via kommandolinje..
VNC (RealVNC)	Fjernskrivebordsprogram — giver et grafisk skrivebord til en headless Raspberry P
Advanced IP Scanner	Scanner et helt IP-range (fx 10.110.0.1–10.110.0.254) og viser hvilke enheder der er på nettet med IP, MAC-adresse og producent — bruges til at finde din Pi/Pico's IP-adresse.

Bageopskrift med PWM-eksempel

Stort set alle opgaver i faget bygger på den samme grundstruktur. Lærer du denne opskrift udenad, kan du løse næsten hvad som helst — du skal blot udskifte "hvad løkken gør" fra opgave til opgave.

1. Importér det du skal bruge

MACHINE giver adgang til hardwaren (ben, PWM, ADC). time giver dig sleep() til at vente mellem handlinger.

```
from machine import Pin, PWM
from time import sleep
```

2. Opret dine pins

Brug pin-layoutet fra afsnit *Pin-layout for Big Beautiful Board*, så du ikke selv skal huske GPIO-numre.

```
ledGL = PWM (Pin (2))
```

3. Sæt PWM-frekvensen

Den interne switching-frekvens, som er typisk 1000 Hz. Sættes kun én gang.

```
led.freq (1000)
```

VÆR OPMÆRKSOM

Dette er kun relevant ved PWM.

4. Skriv selve løkken

Det er her opgaven reelt bliver løst.

```
while True:
    led.duty_u16(65535) # fuldt tændt (100%)
    sleep(0.5)
    led.duty_u16(0) # slukket (0%)
    sleep(0.5)
```

5. Det færdige program

Med try / except kan du stoppe pænt med Ctrl+C, uden at en LED eller PWM-kanal hænger tændt bagefter.

```
from machine import Pin, PWM
from time import sleep

led = PWM(Pin(2))

led.freq(1000)

try:
    while True:
        led.duty_u16(65535)
        sleep(0.5)
        led.duty_u16(0)
        sleep(0.5)

    except KeyboardInterrupt:
        led.duty_u16(0)
        led.deinit()
        print("Stoppet.")
```

Variation 1: brug en knap i stedet for tid

Erstat blot betingelsen i løkken. Husk: PULL_UP betyder tryk = 0.

```
btn_up = Pin(6, Pin.IN, Pin.PULL_UP)

if btn_up.value() == 0: # knap trykket
    led.duty_u16(65535)

else: # knap sluppet
    led.duty_u16(0)
```

Variation 2: flere "faser" (trafiklys, elevator, hysteres)

Har opgaven flere faser der afløser hinanden, erstatter du while-løkken med statemachine-skabelonen (se *Genbrugelige skabeloner*).

VÆR OPMÆRKSOM

Er du i tvivl om, hvor mange states en opgave kræver? Tegn den som cirkler og pile først, så bliver det tydeligt, hvilke funktioner du reelt skal skrive bagefter.

Genbrugelige skabeloner

Disse fem "byggeklodser" dækker næsten alle opgavetyper i faget. Genkend hvilken opgaven kræver, og tilpas skabelonen.

1. Input med validering

Brug den, hver gang opgaven beder brugeren indtaste et tal. Fx indtast en temperatur og vælg etage.

```
def get_int(prompt, low, high):  
    while True:  
        try:  
            value = int(input(prompt))  
  
            if low <= value <= high:  
                return value  
  
            print(f"> Indtast en værdi mellem {low} og {high}:")  
  
        except ValueError:  
            print("> FEJL. Indtast et helt tal.")
```

VÆR OPMÆRKSOM: HVORFOR TRY/EXCEPT HER

int("abc") vil crashe programmet med en ValueError. Ved at fange fejlen kan du bede brugeren prøve igen, i stedet for at programmet stopper.

2. Statemachine

Brug den til trafiklys, elevator, hysteres — alt der har "faser". Hver cirkel i din tegning bliver til én Pythonfunktion.

```
def state_a(): # gør noget  
    sleep(1)  
    return state_b # returnér NÆSTE funktion, kald den ikke selv  
def state_b():  
    sleep(1) return  
state_a state = state_a  
  
while state:  
    state = state()
```

Eksempel 1: Forenklet trafiklys, som statemachine

```
from machine import Pin
from time import sleep

ledRL = Pin(0, Pin.OUT)
ledYL = Pin(1, Pin.OUT)
ledGL = Pin(2, Pin.OUT)

def state_red():
    ledRL.on()
    ledYL.off()
    ledGL.off()
    sleep(2)
    return state_red_yellow

def state_red_yellow():
    ledRL.on()
    ledYL.on()
    ledGL.off()
    sleep(1)
    return state_green

def state_green():
    ledRL.off()
    ledYL.off()
    ledGL.on()
    sleep(2)
    return state_yellow

def state_yellow():
    ledRL.off()
    ledYL.on()
    ledGL.off()
    sleep(1)
    return state_red

state = state_red

try:
    while state:
        state = state()

except KeyboardInterrupt:
    ledRL.off()
    ledYL.off()
    ledGL.off()
```

VÆR OPMÆRKSOM

Læg mærke til hver funktion gør ét stykke arbejde, venter, og returnerer (uden parenteser!) den næste state. Selve while-løkken kalder funktionen for os.

3. ADC og spænding

Standard ADC kode:

```
from machine import ADC, Pin

adc = ADC(Pin(26))

voltage = adc.read_u16() / 65535 * 3.3
```

Skal du måle mere end 3,3 V (batteri, 12 V osv.), skal du bruge en spændingsdelers før Pico'ens ben og gange den rigtige spænding tilbage i koden:

```
real_voltage = adc_voltage / (R2 / (R1 + R2))
```

4. PWM

```
pwm = PWM(Pin(0))
pwm.freq(1000) # fast intern switching-frekvens
pwm.duty_u16(int(pct/100*65535)) # 0-100% lysstyrke
```

5. Knap med PULL_UP

```
button = Pin(6, Pin.IN, Pin.PULL_UP)
if button.value() == 0: # tryk = 0, ikke 1
    ...
```

Eksamen, points og svar

Dette program gav 20 points under først eksamen.

```
from machine import Pin
from time import sleep

button1 = Pin(6, Pin.IN, Pin.PULL_UP)

p1 = Pin(0, Pin.OUT) #red
p0 = Pin(1, Pin.OUT) #yellow

while True:
    if button1.value() == 0:
        p0.on()
        sleep(0.5)
        print("Yellow")
        p0.off()

        p1.on()
        sleep(0.5)
        print("Red")
        p1.off()
```

Den næste del af opgaven var PWM og til eksamen gav det kun 15 points (altså halvdelen, og det fulde antal points er 30 for anden halvdel).

```
#this gave 15 points at the exam!
from machine import Pin, PWM
from time import sleep

#this program gave 15 points doing the exam

#button1 = Pin(6, Pin.IN, Pin.PULL_UP)

led = Pin(0, Pin.OUT)

p1 = machine.Pin(0) #red
led_pwm1 = PWM(p1)

p0 = machine.Pin(1) #yellow
led_pwm2 = PWM(p0)

duty_step = int(input("type a brightness of PMW between 0 to 100%: "))
#duty_step = 129

# Set PWM frequency
freqlim = 20
frequency = int(input("type a frequency between 0 to 20Hz: "))
led_pwm1.freq(frequency)
led_pwm2.freq(frequency)

try:
    while True:

        for duty_cycle in range(0, 65536, duty_step): #65536
            led_pwm1.duty_u16(duty_cycle)
            sleep(0.005)

            led_pwm2.duty_u16(duty_cycle)
            sleep(0.005)

        for duty_cycle in range(65536, 0, -duty_step):
            led_pwm1.duty_u16(duty_cycle)
            sleep(0.005)

            led_pwm2.duty_u16(duty_cycle)
            sleep(0.005)

except KeyboardInterrupt:
    print("Keyboard interrupt")
    led_pwm1.duty_u16(0)
    led_pwm2.duty_u16(0)

    print(led_pwm1)
    print(led_pwm2)
    led_pwm1.deinit()
    led_pwm2.deinit()
```

Med hjælp fra AI har jeg fået ret 15 point version så den lever op til kravene til at få 30 points.

```
from machine import Pin, PWM
from time import sleep

led_pwm1 = PWM(Pin(0)) # red
led_pwm2 = PWM(Pin(1)) # yellow

PWM_CARRIER = 1000 # fixed internal switching freq, used only for dimming
led_pwm1.freq(PWM_CARRIER)
led_pwm2.freq(PWM_CARRIER)

def get_int(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))

            if low <= value <= high:
                return value

            print(f"> Enter a value between {low} and {high}: \n")

        except ValueError:
            print("> ERROR. Please type a whole number.\n")

frequency = get_int("> Enter a frequency between 0 to 20Hz: ", 0, 20)
brightness_pct = get_int("> Enter a brightness between 0-100%: ", 0, 100)

duty = int(brightness_pct / 100 * 65535)

def set_leds(on):
    d = duty if on else 0
    led_pwm1.duty_u16(d)
    led_pwm2.duty_u16(d)

try:
    if frequency == 0:
        set_leds(True) # solid, at whatever brightness was chosen (0% = off)
        while True:
            sleep(1)
    else:
        half_period = (1 / frequency) / 2
        while True:
            set_leds(True)
            sleep(half_period)
            set_leds(False)
            sleep(half_period)
except KeyboardInterrupt:
    led_pwm1.duty_u16(0)
    led_pwm2.duty_u16(0)
    led_pwm1.deinit()
    led_pwm2.deinit()
    print("Stopped, PWM deinitialized.")
```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.

<https://github.com/CN-Thrige/Big-Beautiful-Board>

Ohms lov

Gennemgang af Ohms lov, med formler til hovedregning og lommeregner, samt gennemregnede eksempler.

Ohms lov formler

Modstand i Serieforbindelse

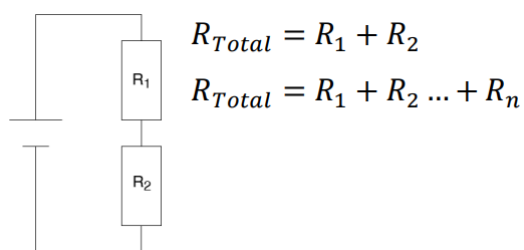


Figure 7: Modstand i Serieforbindelse

Modstand i Parallelforbindelse

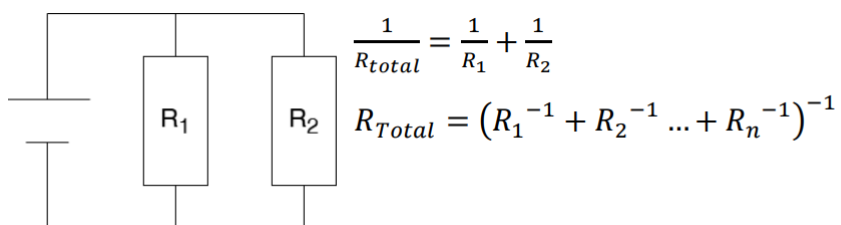
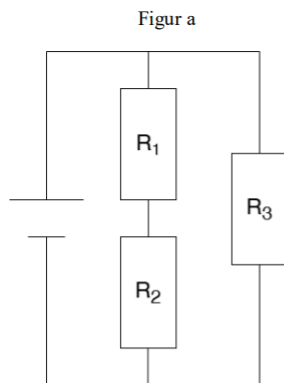


Figure 8: Modstand i Parallelforbindelse

Modstand i Serie- og Parallelforbindelse

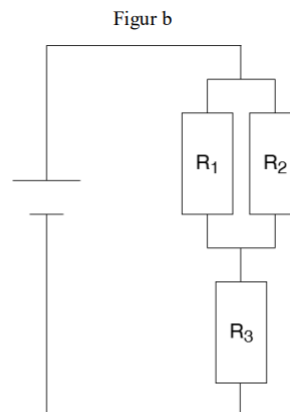


$$R_{1-2} = R_1 + R_2$$

$$R_{Total} = (R_{1-2}^{-1} + R_3^{-1})^{-1}$$

$$R_{Total} = ((R_1 + R_2)^{-1} + R_3^{-1})^{-1}$$

$$R_{Total} = R_1 + R_2 // R_3$$



$$R_{1//2} = (R_1^{-1} + R_2^{-1})^{-1}$$

$$R_{Total} = R_{1//2} + R_3$$

$$R_{Total} = (R_1^{-1} + R_2^{-1})^{-1} + R_3$$

Metode: Når man skal udregne den totale modstand i et kredsløb med både serie- og parallelforbindelser, er det vigtigt at man udregner dem i trin.

I **figur a**, skal man starte med at udregne serieforbindelsen, da den er inde i parallelforbindelsen.

I **figur b**, skal man starte med at udregne parallelforbindelsen, da den er en del af serieforbindelsen.

Figure 9: Modstand i Serie- og Parallelforbindelse

Ohms lovs trekanter



To find the Voltage, (V)

$$[V = I \times R] \quad V \text{ (volts)} = I \text{ (amps)} \times R \text{ (}\Omega\text{)}$$

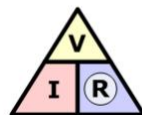
$$\textcircled{V} = I \times R$$



To find the Current, (I)

$$[I = V \div R] \quad I \text{ (amps)} = V \text{ (volts)} \div R \text{ (}\Omega\text{)}$$

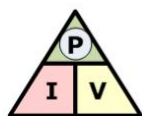
$$\textcircled{I} = \frac{V}{R}$$



To find the Resistance, (R)

$$[R = V \div I] \quad R \text{ (}\Omega\text{)} = V \text{ (volts)} \div I \text{ (amps)}$$

$$\textcircled{R} = \frac{V}{I}$$



To find the Electrical Power (P)

$$[P = V \times I] \quad P \text{ (watts)} = V \text{ (volts)} \times I \text{ (amps)}$$

$$\textcircled{P} = I \times V$$

Figure 10: ohms love trekanter

Oversigt over ohms love

Ohms Lov formler				
Kendte Værdier	Modstand (R)	Strøm (I)	Spænding (U)	Effekt (P)
Strøm & Modstand	---	---	$U = I * R$	$P = I^2 * R$
Volt & Strøm	$R = \frac{U}{I}$	---	---	$P = U * I$
Effekt & Strøm	$R = \frac{P}{I^2}$	---	$U = \frac{P}{I}$	---
Volt & Modstand	---	$I = \frac{U}{R}$	---	$P = \frac{U^2}{R}$
Effekt & Modstand	---	$I = \sqrt{\frac{P}{R}}$	$U = \sqrt{P * R}$	---
Volt & Effekt	$R = \frac{U^2}{P}$	$I = \frac{P}{U}$	---	---

Figure 11: oversigt over ohms lov figur

Præfiks

Præfiks	Navn	10potens	Tal
Z	zetta-	10^{21}	1.000.000.000.000.000.000.000
E	exa-	10^{18}	1.000.000.000.000.000.000
P	peta-	10^{15}	1.000.000.000.000.000
T	tera-	10^{12}	1.000.000.000.000
G	giga-	10^9	1.000.000.000
M	mega-	10^6	1.000.000
k	kilo-	10^3	1.000
h	hecto-	10^2	100
D	deca-	10^1	10
		10^0	1
d	deci-	10^{-1}	0,1 (1/10)
c	centi-	10^{-2}	0,01 (1/100)
m	milli-	10^{-3}	0,001 (1/1000)
μ	micro-	10^{-6}	0,000.001 (1/1.000.000)
n	nano-	10^{-9}	0,000.000.001 (1/1.000.000.000)
p	pico-	10^{-12}	0,000.000.000.001
f	femto-	10^{-15}	0,000.000.000.000.001
a	atto-	10^{-18}	0,000.000.000.000.000.001
z	zepto-	10^{-21}	0,000.000.000.000.000.000.001

Figure 12: præfiks

Enheder til Ohms lov

Symbol	Størrelse	Enhed
U	Spænding	Volt (V)
I	Strøm	Ampere (A)
R	Modstand	Ohm (Ω)

VÆR OPMÆRKSOM: PAS PÅ ENHEDERNE

Formlen kræver hele enheder. altså Ampere, ikke milliampere. Står strømmen i mA, skal du dividere med 1000 først, fx 50 mA = 0,05 A. Samme med modstand i k Ω : 2 k Ω = 2000 Ω .

Hvad siger Ohms lov?

Ohms lov beskriver sammenhængen mellem **spænding**, **modstand** og **strøm** i et elektrisk kredsløb. Jo større modstand, jo mindre strøm løber der ved samme spænding og omvendt.

Tænk på det som vand i et rør: spænding er trykket, der presser vandet igennem, modstand er hvor snævert røret er, og strøm er hvor meget vand der rent faktisk løber igennem pr. sekund. Et smallere rør (større modstand) giver mindre gennemstrømning (mindre strøm) ved samme tryk (spænding).

Ohms lov Formel

$$U = R \times I$$

Da det er en ligning, kan du altid omskrive den, hvis det er strømmen eller modstanden du mangler, i stedet for spændingen

Find spændingen: $U = I \times R$
Find strømmen: $I = U \div R$
Find modstanden: $R = U \div I$

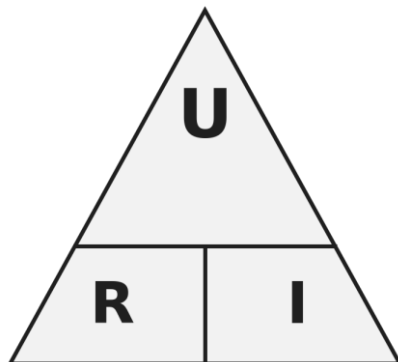


Figure 13: Huskefigur: dæk det, du søger, til med fingeren. De to andre bogstaver viser, om du skal gange eller dividere.

VÆR OPMÆRKSOM: Sådan bruger du trekanten

Dæk U til $\rightarrow$ tilbage står R og I ved siden af hinanden $\rightarrow$ gang dem: $U = R \times I$. Dæk R til $\rightarrow$ U står over I $\rightarrow$ divider: $R = U / I$. Samme princip for I.

Sådan taster du det på lommeregneren

Lommeregneren regner det samme som formlen, du skal bare taste den i rækkefølge fra venstre mod højre:

- Gange (finde U): tast tallet for R, tryk $\times$, tast tallet for I, tryk $=$.
- Dividere (finde I eller R): tast tallet der står øverst i brøken, tryk $\div$, tast tallet der står nederst, tryk $=$.

Eksempler på ohms lov

Eksempel 1: find spændingen (U)

En modstand på $R = 100 \Omega$ har en strøm på $I = 0,5 A$ igennem sig. Hvad er spændingen over modstanden (Find spændingen U)?

Formel:

$$U = R \times I$$

Indsæt tallene:

$$U = 100 \times 0,5$$

Regn ud:

$$U = 50 \text{ V}$$

- På lommeregner: $100 \times 0,5 = 50 \cdot 10^0$ (tjek figur 6 præfiks)

Eksempel 2: find strømmen (I)

Opgave: Der er 12 V over en modstand på $R = 240 \Omega$. Find strømmen I.

Formel:

$$I = U / R$$

Indsæt tallene:

$$I = 12 / 240$$

Regn ud:

$$I = 0,05 \text{ A} = 50 \text{ mA}$$

På lommeregner: $12 \div 240 = 50\text{mA}$

Eksempel 3: find modstanden (R)

Opgave: En LED har 9 V over sig og trækker en strøm på 3 mA. Find modstanden R.

Omregn først mA til A:

$$3 \text{ mA} = 0,003 \text{ A}$$

$$R = U / I$$

Indsæt tallene:

$$R = 9 / 0,003$$

Regn ud:

$$R = 3000 \Omega = 3 \text{ k}\Omega$$

På lommeregner: $9 \div 0.003 = 3 \text{ k}\Omega$

VÆR OPMÆRKSOM:

Glemmer man at omregne mA til A (eller k Ω til Ω) før man regner, bliver svaret 1000 gange forkert. Tjek altid enhederne, før du taster på lommeregneren.

Eksempel 4: Find strømmen (I) 2

Opgave: Et batteri på 12 V er koblet til en modstand på 6 Ω . Hvor stor en strøm løber der i kredsløbet?

Formel:

$$I = U / R$$

Indsæt tallene:

$$I = 12 / 6$$

Regn ud:

$$12/6 = 2 \cdot 10^0 = 2 \text{ A}$$

Eksempler på modstand i serieforbindelse, parallelforbindelse og serie- og parallelforbindelse

Serieforbindelse

Når to eller flere modstande sidder efter hinanden i samme ledningsbane, kaldes det en serieforbindelse. Strømmen har kun én vej at løbe, så den samme strøm løber igennem alle modstandene.

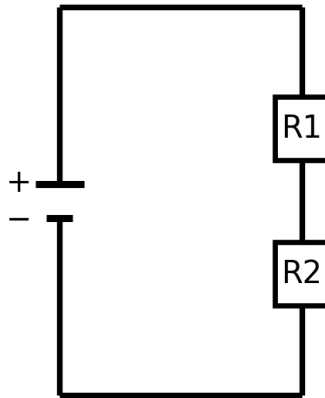


Figure 14: Modstand i serieforbindelse

Serieforbindelse formel:

$$R_{\text{Total}} = R_1 + R_2$$

$$R_{\text{Total}} = R_1 + R_2 \dots + R_n$$

VÆR OPMÆRKSOM

Ved en serieforbindelse lægger du bare modstandene sammen. Den samlede modstand bliver altid større end den største enkeltmodstand.

Eksempel: 2 modstande i serie

Opgave: $R_1 = 10 \, \Omega$ og $R_2 = 20 \, \Omega$ er koblet i serie. Find den samlede modstand R_{Total} .

Formel

$$R_{\text{Total}} = R_1 + R_2$$

Indsæt tallene:

$$R_{\text{Total}} = 10 + 20$$

Regn ud:

$$R_{\text{Total}} = 30 \, \Omega$$

På lommeregner: $10 + 20 = 30 \cdot 10^0$

Modstand i parallelforbindelse

Når to eller flere modstande sidder side om side, så strømmen kan vælge mellem flere veje, kaldes det en parallelforbindelse. Spændingen er den samme over hver modstand, men strømmen deler sig mellem grenene.

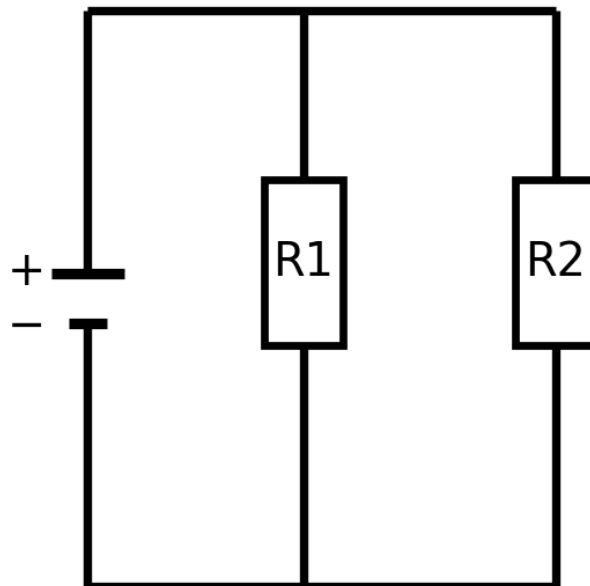


Figure 15: modstand i parallelforbindelse

Formel for parallelforbindelse

$$\frac{1}{R_{\text{Total}}} = \frac{1}{R_1} + \frac{1}{R_2}$$
$$R_{\text{Total}} = (R_1^{-1} + R_2^{-1} \dots + R_n^{-1})^{-1}$$

VÆR OPMÆRKSOM

Når der kun er to modstande i parallel, kan du bruge en meget lettere formel: $R_{\text{Total}} = (R_1 \times R_2) \div (R_1 + R_2)$. Den er nemmere at taste på lommeregner end den generelle formel

Eksempel — to modstande i parallel

Opgave: $R_1 = 10 \, \Omega$ og $R_2 = 20 \, \Omega$ er koblet parallelt. Find den samlede modstand R_{Total} .

Formel:

$$R_{\text{Total}} = (R_1 \times R_2) \div (R_1 + R_2)$$

Indsæt tallene:

$$R_{\text{Total}} = (10 \times 20) \div (10 + 20)$$

Regn ud:

$$R_{\text{Total}} \approx 6,67 \, \Omega$$

På lommeregner: $(10 \times 20) \div (10 + 20) = 6.67 \times 10^0$

VÆR OPMÆRKSOM:

Den samlede modstand ved en parallelforbindelse bliver altid mindre end den mindste enkeltmodstand.

Modstand i serie- og parallelforbindelse

Mange kredsløb er en blanding af serie- og parallelforbindelser. Metoden er at udregne dem i trin, ét stykke ad gangen — start altid med den forbindelse, der sidder "inde i" den anden.

Figur a: serieforbindelse inde i en parallelforbindelse

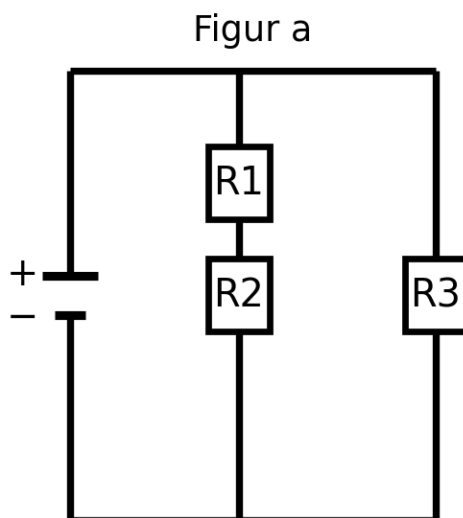


Figure 16: Figur a, Serieforbindelse inde i en parallelforbindelse

R_1 og R_2 sidder i serie med hinanden, og denne kombination sidder parallelt med R_3 . Her skal du starte med serieforbindelsen ($R_1 + R_2$), fordi den sidder inde i parallelforbindelsen.

Formel:

$$R_1 + R_2 = R_1 + R_2$$

$$R_{\text{Total}} = (R_1 + R_2 \times R_3) \div (R_1 + R_2 + R_3)$$

Eksempel: figur a med tal

Opgave: $R_1 = 10 \Omega$ og $R_2 = 20 \Omega$ sidder i serie, og denne kombination sidder parallelt med $R_3 = 15 \Omega$. Find R_{Total} .

Trin 1: udregn seriedelen først (den sidder inde i parallelforbindelsen):

$$R_{1+2} = R_1 + R_2 = 10 + 20 = 30 \Omega$$

Trin 2: brug parallel-genvejen på R_{1+2} og R_3 :

$$R_{\text{Total}} = (R_{1+2} \times R_3) \div (R_{1+2} + R_3)$$

$$R_{\text{Total}} = (30 \times 15) \div (30 + 15)$$

$$R_{\text{Total}} = 10 \Omega$$

På lommeregner: $(30 \times 15) \div (30 + 15) = 10 \times 10^0$

Figur b: parallelforbindelse inde i en serieforbindelse

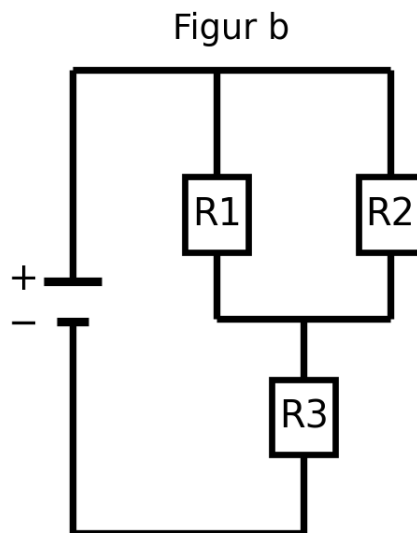


Figure 17. Figur b, parallelforbindelse inde i en serieforbindelse

R_1 og R_2 sidder parallelt med hinanden, og denne kombination sidder i serie med R_3 . Her skal du starte med parallelforbindelsen (R_1 og R_2), fordi den er en del af serieforbindelsen.

$$R_1 // R_2 = (R_1 \times R_2) \div (R_1 + R_2)$$
$$R_{\text{Total}} = R_1 // R_2 + R_3$$

Sådan taster du Ohms lov ind på lommeregneren

Ohms lov og modstandsformlerne kræver kun gange ($\times$), division ($\div$) og plus ($+$), så du behøver ikke en avanceret lommeregner. Et par gode råd:

- Brug punktum eller komma til decimaltal, alt efter hvilken lommeregner du bruger (fx 0,5 eller 0.5).
- Tast tallene i samme rækkefølge, som de står i formlen: først tallet, så regnetegnet, så det andet tal, og til sidst "=".
- Ved brøker (fx parallelformlen): tast altid det, der står i parentes, færdigt først, før du dividerer med den anden parentes.
- Dobbelttjek altid enheden på dit svar: Volt (V) til spænding, Ampere (A) til strøm, og Ohm (Ω) til modstand.